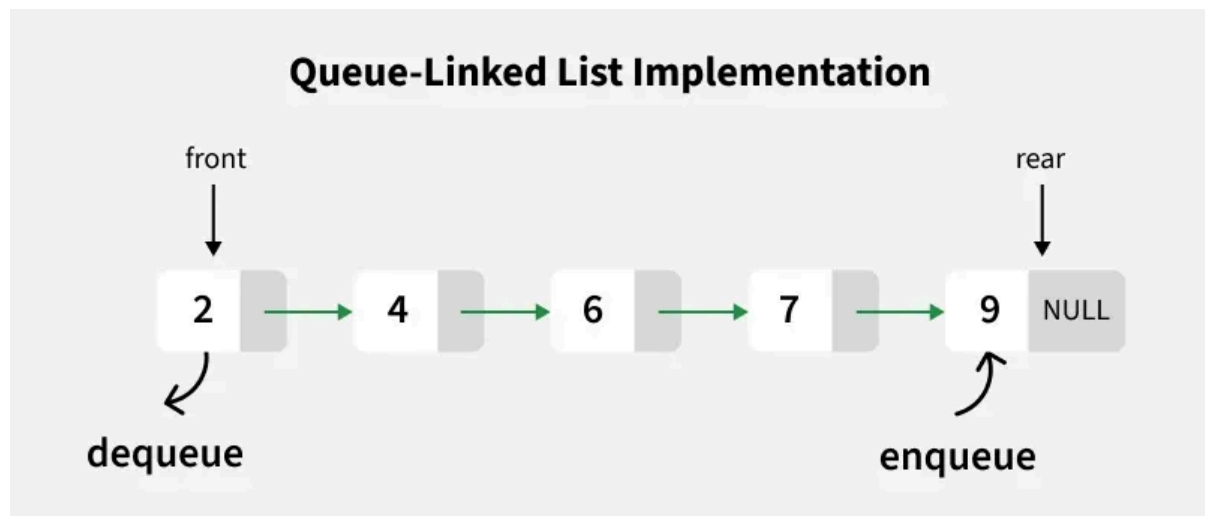


Queue - Linked List Implementation

Last Updated : 20 Sep, 2025

A Queue is a linear data structure that follows the **First-In-First-Out (FIFO)** principle. The element inserted first is the first one to be removed.



It can be implemented using a linked list, where each element of the queue is represented as a node.

Declaration of Queue using Linked List

To implement a queue with a linked list, we maintain:

A Node structure/class that contains:

- data → to store the element.
- next → pointer/reference to the next node in the queue.

Two pointers/references:

- front → points to the first node (head of the queue).
- rear → points to the last node (tail of the queue).

[C++](#)[C](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
class Node {
    int data;
    Node next;
    Node(int new_data) {
        data = new_data;
        next = null;
    }
}

// Queue class definition
class myQueue {
    private Node front;
    private Node rear;

    public myQueue() {
        front = rear = null;
    }
};
```



Operations on Queue using Linked List:

Enqueue Operation

The enqueue operation adds an element to the rear of the queue. Unlike array implementation, there is no fixed capacity in linked list. Overflow occurs only when memory is exhausted.

- A new node is created with the given value.
- If the queue is empty (`front == null` and `rear == null`), both front and rear are set to this new node.
- Otherwise, the current rear's next pointer is set to the new node.
- The rear pointer is updated to point to the new node.

[C++](#)[C](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
public void enqueue(int new_data) {
    Node new_node = new Node(new_data);
    if (isEmpty()) {
        front = rear = new_node;
    } else {
        rear.next = new_node;
        rear = new_node;
    }
}
```



Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Deque Operation

The dequeue operation removes an element from the front of the queue.

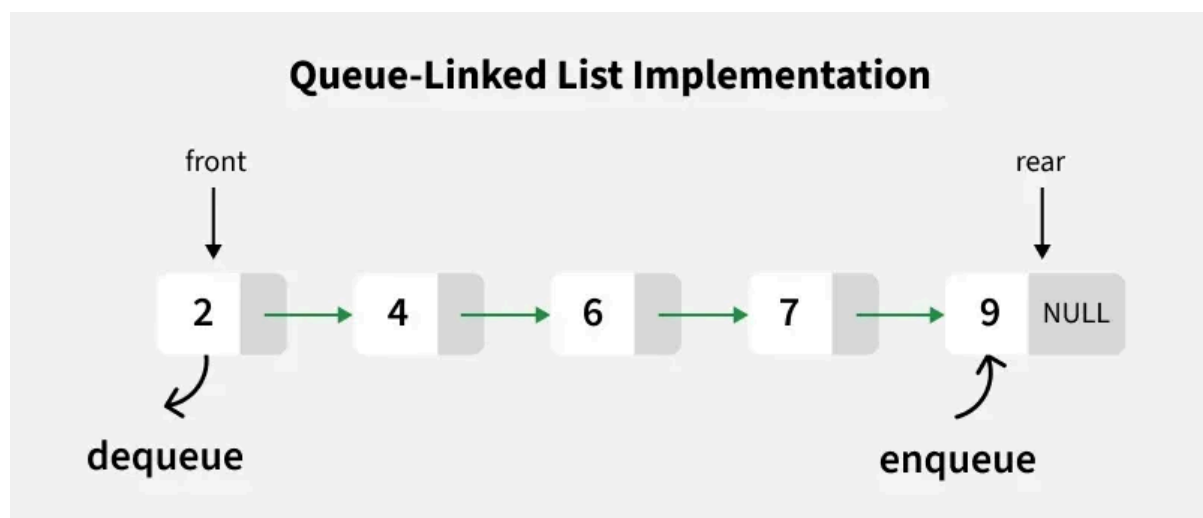
- If the queue is empty ($\text{front} == \text{null}$), return underflow (queue is empty).
- Otherwise, store the current front node in a temporary pointer.
- Move the front pointer to the next node ($\text{front} = \text{front.next}$).
- If the front becomes null, also set $\text{rear} = \text{null}$ (queue becomes empty).

[C++](#)[C](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
public void dequeue() {  
    if (isEmpty()) {  
        System.out.println("Queue Underflow");  
        return;  
    }  
  
    Node temp = front;  
    front = front.next;  
    if (front == null) rear = null;  
    temp = null;  
}
```

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$



isEmpty Operation

The isEmpty operation checks whether the queue has no elements.

- If the front pointer is NULL, it means the queue is empty → return true.
- Otherwise, the queue has elements → return false.

C++

C

Java

Python

C#

JavaScript

```
// Check if the queue is empty
public boolean isEmpty() {
    return front == null;
}
```



Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Front Operation

The front() function returns the element at the front of the queue without removing it.

- If the queue is empty (front == NULL), print a message and return -1.
- Otherwise, return front->data (the value at the front).

C++

C

Java

Python

C#

JavaScript

```
// Return front element
public int getfront() {
    if (isEmpty()) {
        System.out.println("Queue is empty");
        return -1;
    }
    return front.data;
}
```



Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Full Implementations of Queue Using Linked List

C++

C

Java

Python

C#

JavaScript

```
// Node class definition
class Node {
```



```
int data;
Node next;
Node(int new_data) {
    data = new_data;
    next = null;
}
}

// Queue class definition
class myQueue {
    private Node front;
    private Node rear;
```

[Review Questions](#) [Quizzes](#) [Must Do](#) [Advanced DSA](#) [System Design](#) [Aptitude](#)

[Sign In](#)

```
public myQueue() {
    currSize = 0;
    front = rear = null;
}

// Check if the queue is empty
public boolean isEmpty() {
    return front == null;
}

// Enqueue operation
public void enqueue(int new_data) {
    Node new_node = new Node(new_data);
    if (isEmpty()) {
        front = rear = new_node;
    } else {
        rear.next = new_node;
        rear = new_node;
    }

    currSize++;
}

// Dequeue operation (returns removed element)
public int dequeue() {
    if (isEmpty()) {
        System.out.println("Queue Underflow");
        return -1;
    }
}
```

```
        Node temp = front;
        int removedData = temp.data;
        front = front.next;
        if (front == null) rear = null;

        currSize--;
        return removedData;
    }

    // Return front element
    public int getfront() {
        if (isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        return front.data;
    }

    // Return size of the queue
    public int size() {
        return currSize;
    }
}

class GFG {
    public static void main(String[] args) {
        myQueue q = new myQueue();

        q.enqueue(10);
        q.enqueue(20);

        System.out.println("Dequeue: " + q.dequeue());

        q.enqueue(30);

        System.out.println("Front: " + q.getfront());
        System.out.println("Size: " + q.size());
    }
}
```

Output

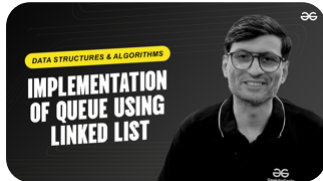
Dequeue: 10

Front: 20

Size: 2

Related Article:

[Array Implementation of Queue](#)



Imple
of Que
Linked



Linked
Imple
of Que
Pythor



Implementation of Queue using Linked List

[Visit Course](#)[Comment](#)[More info](#)

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
GATE
Software and Tools

Explore

POTD
Job-A-Thon
Community
Blogs
Nation Skill Up

Courses

IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Aptitude
Puzzles
GfG 160
DSA 360
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved